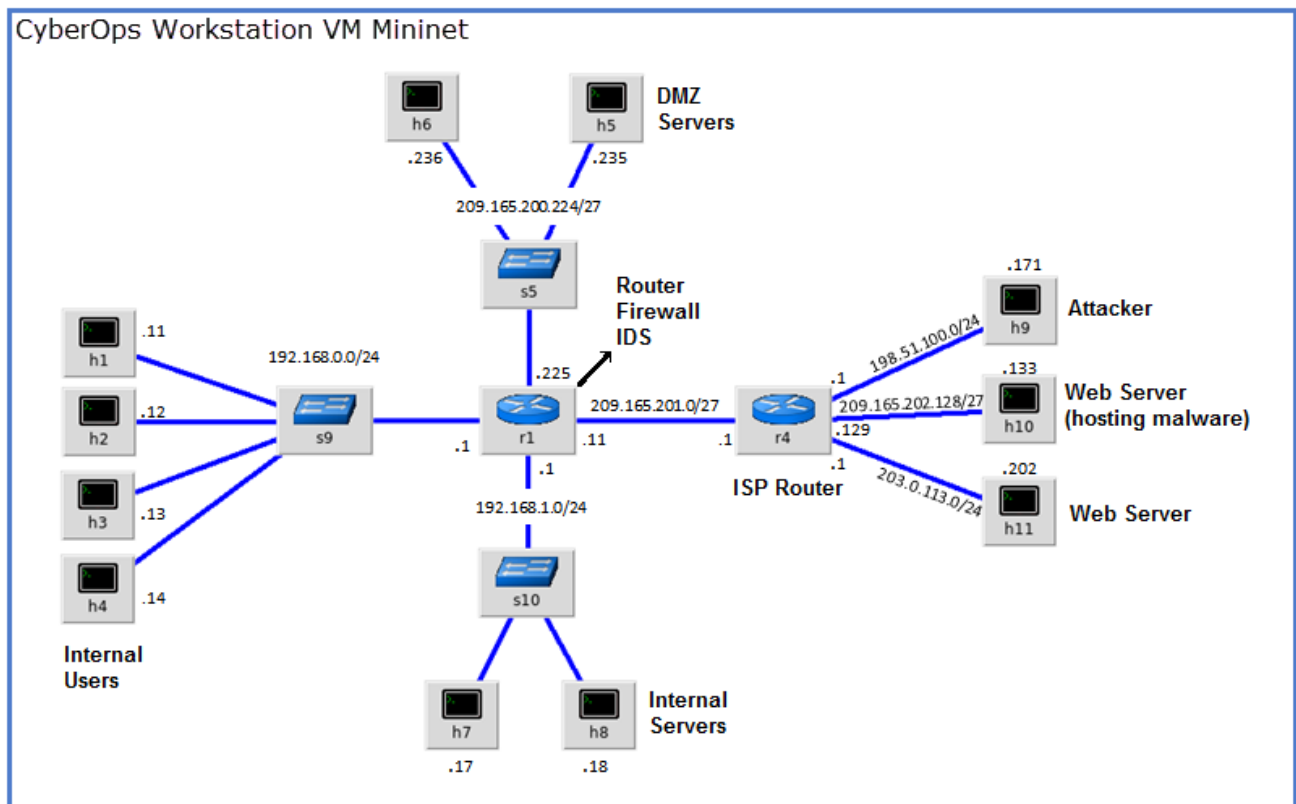


Firewall, VPN's, IPS, Endpoint

Assignment o3

Network Alerts

Topology



Required Resources

- CyberOps Workstation virtual machine
- Internet connection
- **Note:** In this lab, the CyberOps Workstation VM is a container for holding the Mininet environment shown in the Topology. If a memory error is received in an attempt to run any command,

quit out of the step, go to the VM settings, and increase the memory. The default is 1 GB; try 2GB.

Instructions

Part 1: Preparing the Virtual Environment

Launch Oracle VirtualBox and change the CyberOps Workstation for Bridged mode, if necessary. Select **Machine** > **Settings** > **Network**. Under Attached To, select Bridged Adapter (or if you are using WiFi with a proxy, you may need NAT adapter) and click OK.

Launch the CyberOps Workstation VM, open a terminal and configure its network by executing the *configure_as_dhcp.sh script*. Because the script requires super-user privileges, provide the password for the user analyst.

```
[analyst@secOps ~]$ sudo ./lab.support.files/scripts/configure_as_dhcp.sh
[sudo] password for analyst:
[analyst@secOps ~]$
```

Use the `ifconfig` command to verify CyberOps Workstation VM now has an IP address on your local network. You can also test connectivity to a public webserver by pinging `www.cisco.com`. Use Ctrl+C to stop the pings.

Part 2: Firewall and IDS Logs

From the CyberOps Workstation VM, run the script to start mininet.

```
[analyst@secOps ~]$ sudo
./lab.support.files/scripts/cyberops_extended_topo_no_fw.py
```

The mininet prompt should be displayed, indicating mininet is ready for commands.

From the mininet prompt, open a shell on R1 using the command below:

```
mininet> xterm R1
mininet>
```

Question:

The R1 shell opens in a terminal window with black text and white background. What user is logged into that shell? What is the indicator of this?

Instead of “[analyst@secOps ~]”, it is now “[root@secOps analyst]#” meaning that the root user is now logged in. The # is an indicator of enabled administrator privileges.

From R1’s shell, start the Linux-based IDS, Snort.

```
[root@secOps analyst]# ./lab.support.files/scripts/start_snort.sh
```

Note: You will not see a prompt as Snort is now running in this window. If for any reason, Snort stops running and the [root@secOps analysts]# prompt is displayed, rerun the script to launch Snort. Snort must be running to capture alerts later in the lab.

From the CyberOps Workstation VM mininet prompt, open shells for hosts H5 and H10.

```
mininet> xterm H5
mininet> xterm H10
```

H10 will simulate a server on the Internet that is hosting malware. On H10, run the mal_server_start.sh script to start the server.

```
[root@secOps analyst]# ./lab.support.files/scripts/mal_server_start.sh
```

On H10, use netstat with the -tunpa options to verify that the web server is running. When used as shown below, netstat lists all ports currently assigned to services:

```
[root@secOps analyst]# netstat -tunpa
```

```
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address Foreign Address State
PID/Program name
tcp    0      0 0.0.0.0:6666      0.0.0.0:*      LISTEN
1839/nginx: master
```

```
[root@secOps analyst]#
```

As seen by the output above, the lightweight webserver nginx is running and listening to connections on port TCP 6666.

In the R1 terminal window, an instance of Snort is running. To enter more commands on R1, open another R1 terminal by entering the `xterm R1` again in the CyberOps Workstation VM terminal window. You may also want to arrange the terminal windows so that you can see and interact with each device.

In the new R1 terminal tab, run the `tail` command with the `-f` option to monitor the `/var/log/snort/alert` file in real-time. This file is where snort is configured to record alerts.

```
[root@secOps analyst]# tail -f /var/log/snort/alert
```

Because no alerts were yet recorded, the log should be empty. However, if you have run this lab before, old alert entries may be shown. In either case, you will not receive a prompt after typing this command. This window will display alerts as they happen.

From H5, use the `wget` command to download a file named `W32.Nimda.Amm.exe`. Designed to download content via HTTP, `wget` is a great tool for downloading files from web servers directly from the command line.

```
[root@secOps analyst]# wget 209.165.202.133:6666/W32.Nimda.Amm.exe
```

Questions:

What port is used when communicating with the malware web server? What is the indicator?

The output from `netstat -tunpa` shows the address “0.0.0.0:6666” which indicates that Port 6666 is active in a listening state.

Was the file completely downloaded?

Yes.

Did the IDS generate any alerts related to the file download?

The alert log gave an alert stating “Malicious Server Hit!” as the download command was given, as shown below.

As the malicious file was transiting R1, the IDS, Snort, was able to inspect its payload. The payload matched at least one of the signatures configured in Snort and triggered an alert on the second R1 terminal window (the tab where tail -f is running). The alert entry is shown below. Your timestamp will be different:

```
04/28-17:00:04.092153 [**] [1:1000003:0] Malicious Server Hit! [**]  
[Priority: 0] {TCP} 209.165.200.235:34484 -> 209.165.202.133:6666
```

Questions:

Based on the alert shown above, what was the source and destination IPv4 addresses used in the transaction?

Source: 209.165.200.235:34484 -> Destination: 209.165.202.133:6666

Based on the alert shown above, what was the source and destination ports used in the transaction?

Source: 34484 -> Destination: 6666

Based on the alert shown above, when did the download take place?

04/28-17:00:04.092153

The download took place four seconds and 092154 milliseconds into 5PM on April 4th, 2017.

Based on the alert shown above, what was the message recorded by the IDS signature?

Malicious Server Hit!

On H5, use the tcpdump command to capture the event and download the malware file again so you can capture the transaction. Issue the following command below start the packet capture:

```
[root@secOps analyst]# tcpdump -i H5-eth0 -w nimda.download.pcap &  
[1] 5633  
[root@secOps analyst]# tcpdump: listening on H5-eth0, link-type EN10MB  
(Ethernet), capture size 262144 bytes
```

The command above instructs tcpdump to capture packets on interface H5-eth0 and save the capture to a file named nimda.download.pcap.

The & symbol at the end tells the shell to execute tcpdump in the background. Without this symbol, tcpdump would make the terminal unusable while it was running. Notice the [1] 5633; it indicates one process was sent to background and its process ID (PID) is 5366. Your PID will most likely be different.

Press ENTER a few times to regain control of the shell while tcpdump runs in background.

Now that tcpdump is capturing packets, download the malware again. On H5, re-run the command or use the up arrow to recall it from the command history facility.

```
[root@secOps analyst]# wget 209.165.202.133:6666/W32.Nimda.Amm.exe
```

Stop the capture by bringing tcpdump to foreground with the fg command. Because tcpdump was the only process sent to background, there is no need to specify the PID. Stop the tcpdump process with Ctrl+C. The tcpdump process stops and displays a summary of the capture. The number of packets may be different for your capture.

```
[root@secOps analyst]# fg
tcpdump -i h5-eth0 -w nimda.download.pcap
^C316 packets captured
316 packets received by filter
0 packets dropped by kernel
[root@secOps analyst]#
```

On H5, Use the ls command to verify the pcap file was in fact saved to disk and has size greater than zero.

Question:

How can be this PCAP file be useful to the security analyst?

This PCAP file can be analyzed with tcpdump or Wireshark to determine how the hosts were communicating. It gives the source and destination IP addresses, the domains, URLs, and paths of potentially harmful files, and the use of ports in itself can help determine the nature of a communication.

Tuning Firewall Rules Based on IDS Alerts

In the CyberOps Workstation VM, start a third R1 terminal window.
mininet > xterm R1

In the new R1 terminal window, use the iptables command to list the chains and their rules currently in use:

```
[root@secOps ~]# iptables -L -v
```

Question:

What chains are currently in use by R1?

None.

Connections to the malicious server generate packets that must transverse the iptables firewall on R1. Packets traversing the firewall are handled by the FORWARD rule and therefore, that is the chain that will receive the blocking rule. To keep user computers from connecting to the malicious server identified in Step 1, add the following rule to the FORWARD chain on R1:

```
[root@secOps ~]# iptables -I FORWARD -p tcp -d 209.165.202.133 --dport 6666  
-j DROP  
[root@secOps ~]#
```

Where:

- -I FORWARD: inserts a new rule in the FORWARD chain.
- -p tcp: specifies the TCP protocol.
- -d 209.165.202.133: specifies the packet's destination
- --dport 6666: specifies the destination port
- -j DROP: set the action to drop.

Use the iptables command again to ensure the rule was added to the FORWARD chain. The CyberOps Workstation VM may take a few seconds to generate the output:

```
[root@secOps analyst]# iptables -L -v
```

On H5, try to download the file again:

```
[root@secOps analyst]# wget 209.165.202.133:6666/W32.Nimda.Amm.exe
```

Enter Ctrl+C to cancel the download, if necessary.

Question:

Was the download successful this time? Explain.

No, the H5 node was unable to connect to the address after the firewall rule was added.

What would be a more aggressive but also valid approach when blocking the offending server?

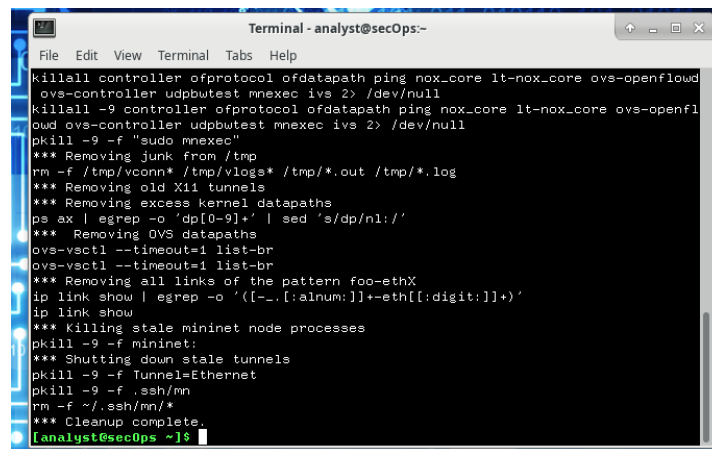
By using two commands, “iptables -I INPUT -s 209.165.202.133 -j DROP” and “iptables -I OUTPUT -d 209.165.202.133 -j DROP” can prevent packets from being sent or received from that IP.

Part 3: Terminate and Clear Mininet Process

Navigate to the terminal used to start Mininet. Terminate the Mininet by entering quit in the main CyberOps VM terminal window.

After quitting Mininet, clean up the processes started by Mininet. Enter the password cyberops when prompted.

```
[analyst@secOps scripts]$ sudo mn -c  
[sudo] password for analyst:
```



```
Terminal - analyst@secOps:-  
File Edit View Terminal Tabs Help  
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd  
ovs-controller udptest mnexec ivs 2> /dev/null  
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openfl  
owd ovs-controller udptest mnexec ivs 2> /dev/null  
pkill -9 -f "sudo mnexec"  
*** Removing junk from /tmp  
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log  
*** Removing old X11 tunnels  
*** Removing excess kernel datapaths  
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'  
*** Removing OVS datapaths  
ovs-vsctl --timeout=1 list-br  
ovs-vsctl --timeout=1 list-br  
*** Removing all links of the pattern foo-ethX  
ip link show | egrep -o '([[:alnum:]]+-eth[[:digit:]]+)'  
ip link show  
*** Killing stale mininet node processes  
pkill -9 -f mininet:  
*** Shutting down stale tunnels  
pkill -9 -f Tunnel=Ethernet  
pkill -9 -f .ssh/mn  
rm -f ~/.ssh/mn/*  
*** Cleanup complete.  
[analyst@secOps ~]$
```